Exercise No: 2	Securing VPC Resources by Using Security Groups
Date of Exercise	21.08.2026

AIM:

To understand and implement network security controls in Amazon Web Services (AWS) by configuring security groups and network access control lists (ACLs) within an Amazon Virtual Private Cloud (VPC). To examine subnet routing, restrict network traffic based on IP addresses and security group references, establish secure connections to instances in private subnets using SSH through a bastion host, and access private EC2 instances using AWS Systems Manager Session Manager.

DESCRIPTION:

This lab focused on securing resources within an Amazon Virtual Private Cloud (VPC) by examining and configuring security groups and network access control lists (ACLs). The existing network architecture, including public and private subnets, route tables, a NAT Gateway, an Internet Gateway and EC2 instances, was examined to understand how network traffic was managed.

The activities involved testing HTTP connectivity between public proxy servers and a private application server, restricting access through specific IP addresses and security group references, and controlling traffic using network ACL rules. Secure access to the private AppServer was also established through a bastion host using SSH agent forwarding. Additionally, AWS Systems Manager Session Manager was used to connect directly to the private instance and modify its hosted webpage without requiring a bastion host.

PROCEDURE:

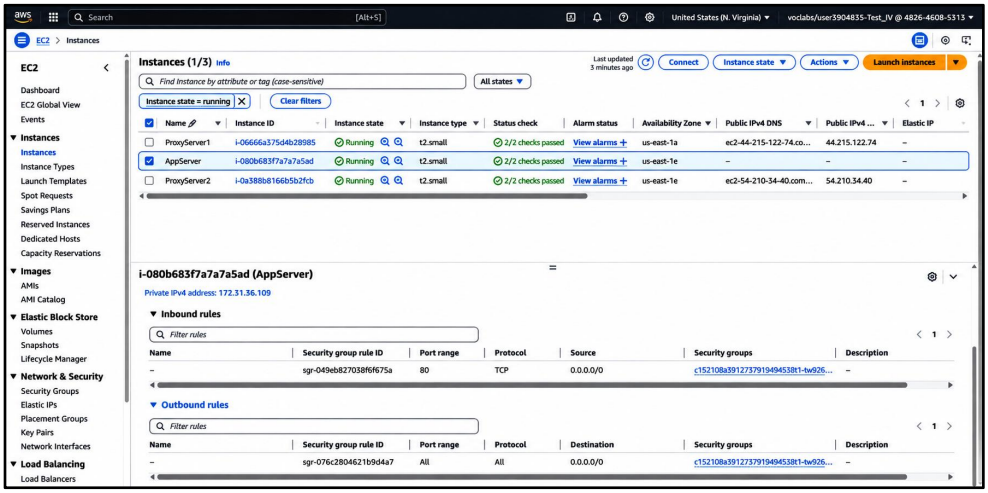
Task 1: Analyzing the VPC and private subnet resource settings

The Amazon VPC configuration was examined to understand the network structure and routing of private subnet resources. The LabVPC CIDR range was identified as 10.0.0.0/16, and its route table was inspected to verify that local traffic was routed within the VPC while external traffic was directed through the NAT Gateway. The route table was renamed from changeme to Private.

The AppServer instance was then examined to verify that it was deployed in the PrivateSubnet and had only a private IPv4 address. The associated AppServerSG was inspected to identify its inbound HTTP rule, which allowed TCP traffic on port 80 from 0.0.0.0/0. Its outbound rules and stateful behavior were also reviewed.

Result: The VPC, private subnet routing configuration, AppServer instance, and associated security group settings were successfully examined.

OUTPUT:



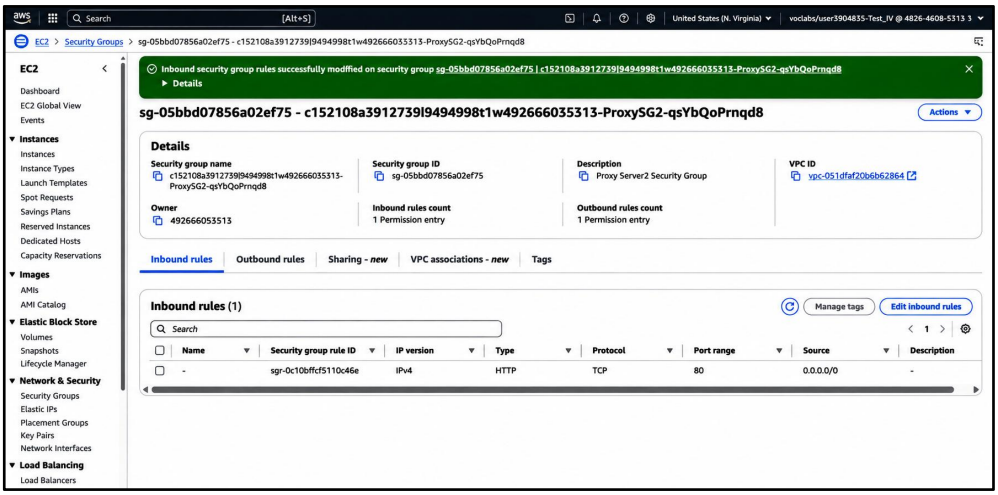
Task 2: Analyzing the public subnet resource settings

The public subnet configurations within the LabVPC were examined to understand how internet connectivity was provided to the EC2 instances. The route table associated with PublicSubnetA was inspected to verify that external traffic was routed through the Internet Gateway attached to the VPC.

The ProxyServer1 instance in PublicSubnetA was examined to confirm that it had a public IPv4 address and was associated with the ProxySG security group, which allowed inbound HTTP traffic on TCP port 80 from any IPv4 address. Similarly, ProxyServer2 in PublicSubnetB was inspected, and an inbound HTTP rule was added to its ProxySG2 security group to allow traffic from Anywhere-IPv4.

Result: The PublicSubnetA route to the Internet Gateway and the configurations of ProxyServer1 and ProxyServer2 were examined. The ProxySG2 security group was updated to allow inbound HTTP traffic from Anywhere-IPv4, enabling both proxy servers to be accessed over HTTP.

OUTPUT:



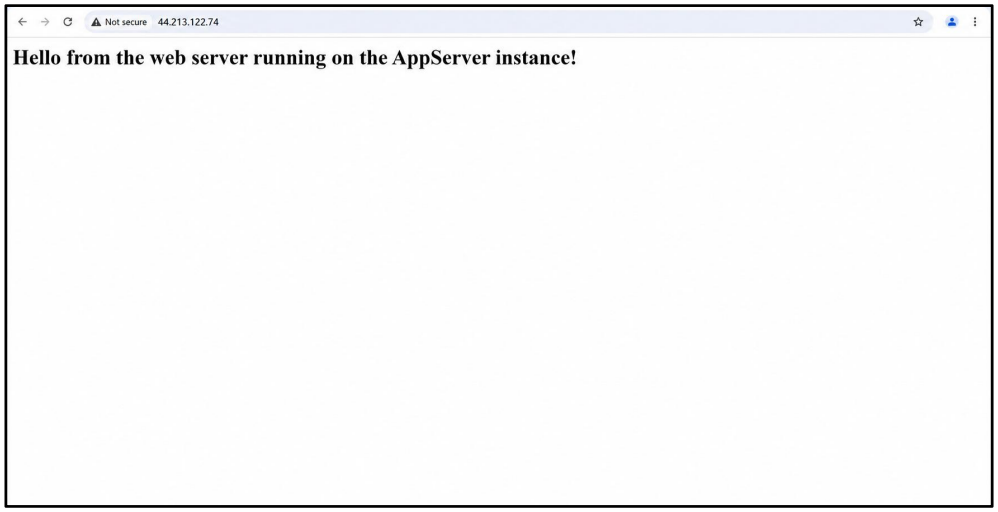
Task 3: Testing HTTP connectivity from public EC2 instances

The HTTP connectivity of the public EC2 instances was tested by accessing their respective public IPv4 addresses through a web browser. ProxyServer1 and ProxyServer2 were accessed separately to verify whether they could communicate with the private AppServer through the configured proxy forwarding mechanism.

Both proxy servers successfully displayed the webpage hosted on the AppServer, confirming that HTTP requests were being forwarded correctly from the public instances to the private application server.

Result: HTTP connectivity was successfully established from both ProxyServer1 and ProxyServer2 to the private AppServer.

OUTPUT:



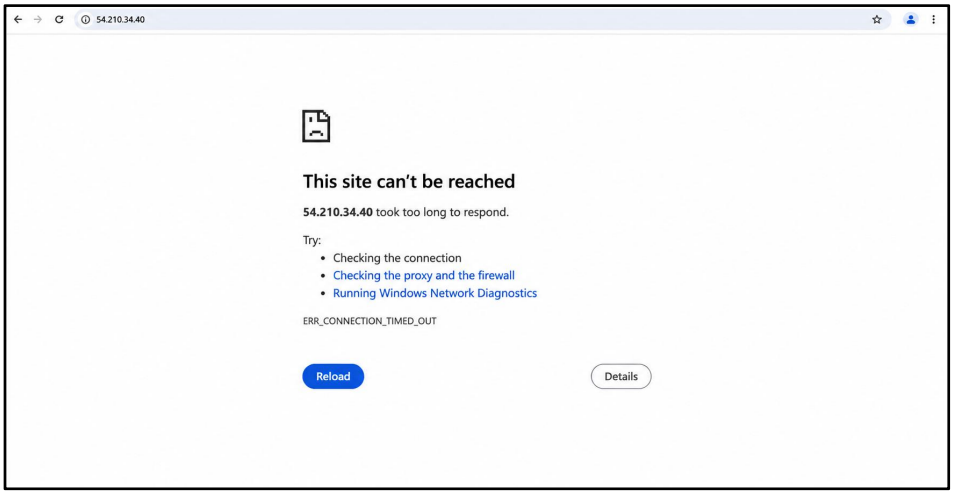
Task 4: Restricting HTTP access by using an IP address

The inbound HTTP rule of the AppServerSG security group was modified to restrict access to the private AppServer. The existing source address 0.0.0.0/0 was replaced with the private IPv4 address of ProxyServer1 using the /32 CIDR notation. This limited HTTP access to the specified proxy server.

The connectivity was tested again through both public proxy servers. ProxyServer1 continued to load the AppServer webpage successfully, whereas ProxyServer2 could no longer access it and the connection timed out.

Result: HTTP access to the AppServer was successfully restricted to ProxyServer1 by specifying its private IP address.

OUTPUT:



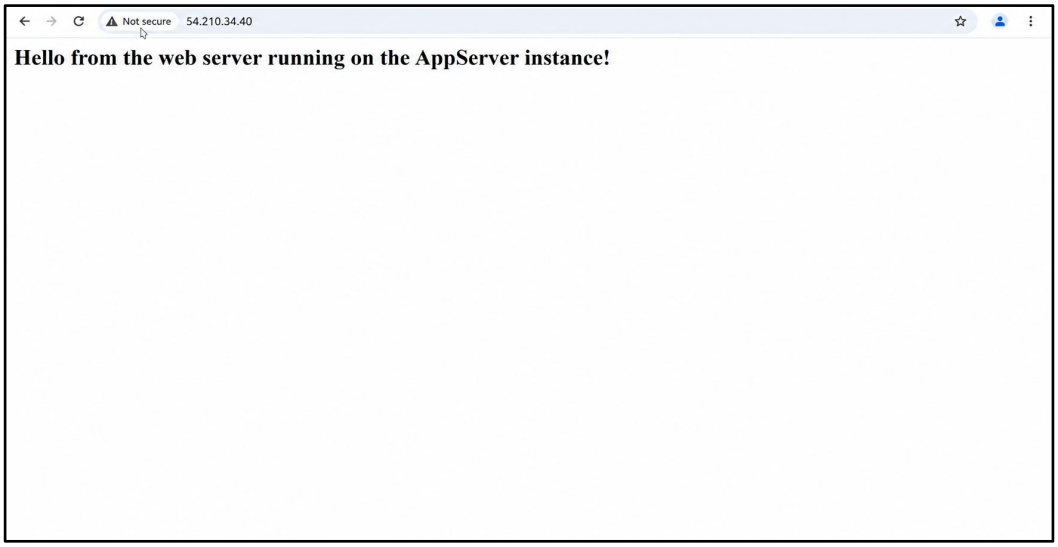
Task 5: Scaling restricted HTTP access by referencing a security group

The inbound HTTP rule of the AppServerSG security group was modified to use a security group reference instead of a specific IP address. The existing IP-based source restriction was replaced with the ProxySG security group, allowing instances associated with that security group to access the AppServer over HTTP.

Next, the security group associated with ProxyServer2 was changed from ProxySG2 to ProxySG. Connectivity was tested again from both proxy servers to verify the updated access configuration.

Result: Both ProxyServer1 and ProxyServer2 successfully accessed the AppServer using a security group reference, demonstrating a scalable approach to managing network access.

OUTPUT:



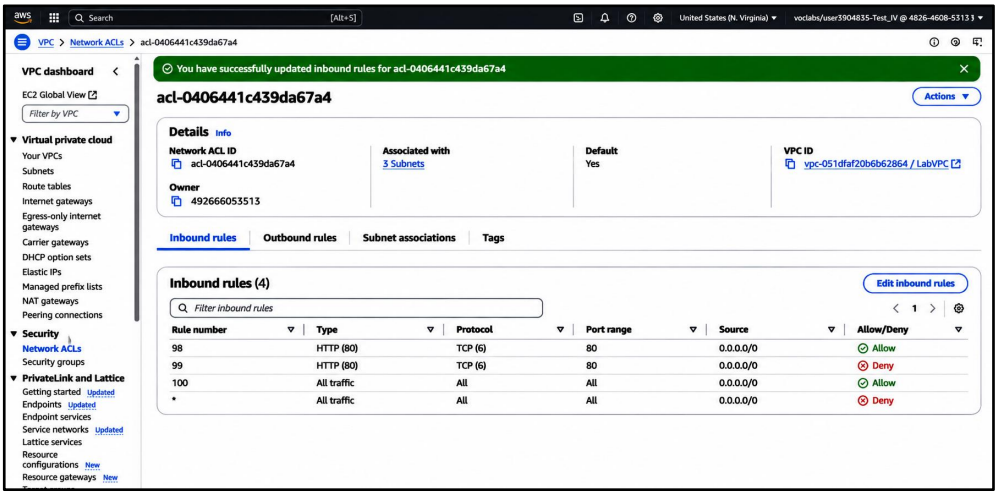
Task 6: Restricting HTTP access by using a network ACL

The network ACL associated with the subnet was configured to demonstrate how HTTP traffic could be controlled using inbound rules. An inbound rule with rule number 99 was added to deny HTTP traffic on TCP port 80. After applying the rule, connectivity to the AppServer through ProxyServer1 was tested, and the connection timed out.

To restore access, another inbound rule with rule number 98 was added to allow HTTP traffic. The connectivity test was repeated, and the AppServer webpage loaded successfully. This demonstrated that network ACL rules are evaluated in ascending order, with the first matching rule taking precedence.

Result: HTTP traffic was successfully blocked and subsequently allowed by modifying network ACL rules and their evaluation priority.

OUTPUT:



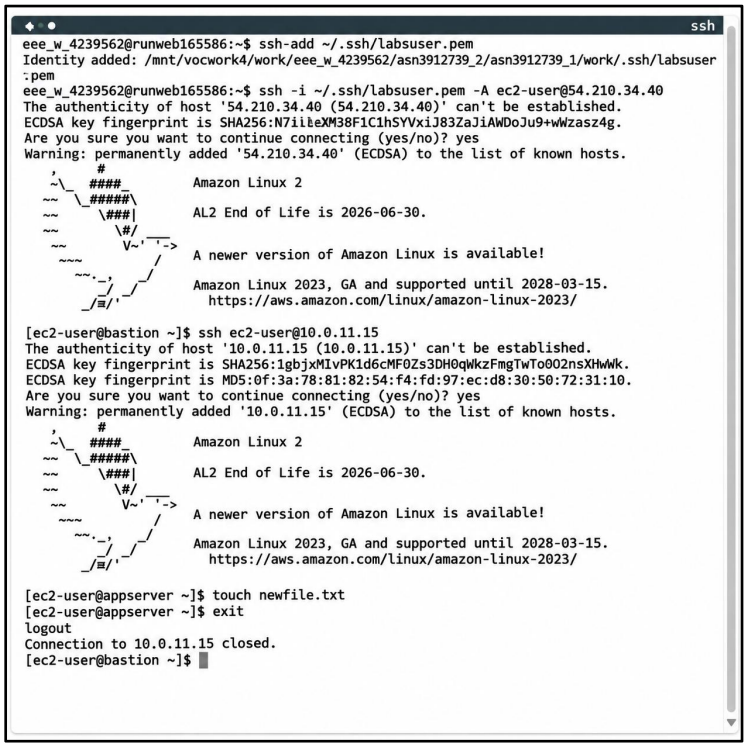
Task 7: Connecting to the AppServer by using a bastion host and SSH

The ProxyServer2 instance was renamed to Bastion and configured to act as a bastion host for accessing the private AppServer. A new security group named BastionSG was created in the LabVPC, with an inbound rule allowing SSH traffic on TCP port 22 from Anywhere-IPv4. This security group was associated with the Bastion instance. The AppServerSG was also updated to allow SSH traffic from the Bastion host's private IP address using /32 CIDR notation.

SSH agent forwarding was configured using ssh-agent and ssh-add with the provided private key. An SSH connection was established to the Bastion host, followed by another SSH connection to the AppServer using its private IP address. A file named newfile.txt was created on the AppServer to verify successful access. Agent forwarding allowed authentication without storing the private key on the Bastion host.

Result: The private AppServer was successfully accessed through the Bastion host using SSH agent forwarding, and a file was created to verify the connection.

OUTPUT:



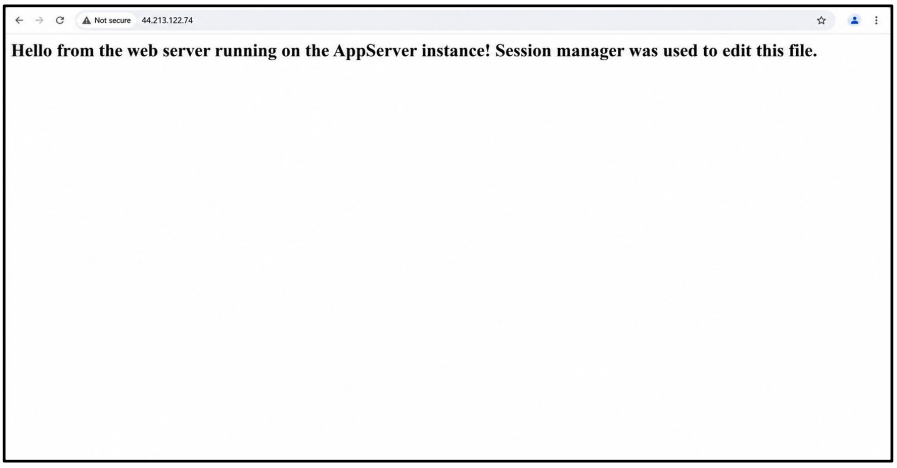
Task 8: Connecting directly to a host in a private subnet by using Session Manager

The AppServer instance was accessed directly through AWS Systems Manager Session Manager. The EC2 console was opened, the AppServer instance was selected, and a Session Manager connection was initiated. This provided a command-line session to the private instance without requiring SSH access through a bastion host or opening inbound SSH port 22.

After establishing the session, the hosted webpage was modified using the sed command to add a message indicating that Session Manager was used to edit the file. The updated webpage was then verified by accessing the application through ProxyServer1's public IPv4 address.

Result: The private AppServer was successfully accessed using Session Manager, and its hosted webpage was modified and verified without requiring a bastion host.

OUTPUT:



BADGE:



PRESENTATION LINK:

[Insert YouTube / presentation link]

RESULT:

The lab demonstrated how security groups and network ACLs control network traffic within an Amazon VPC. HTTP access to a private application server was tested and restricted using IP addresses, security group references, and ACL rule priorities. Secure access to the private AppServer was also established through a bastion host using SSH agent forwarding and directly through AWS Systems Manager Session Manager. These activities demonstrated practical methods of securing communication between public and private resources.